

Sieci Komputerowe



⌘ Wykład:

Protokoły Internetu

Materiały do Laboratoriów:

TRAFFIC

Protokół IP



⌘ Protokół IP zawiera trzy definicje:

- ☒ Definicję podstawowej jednostki przesyłanych danych, używanej w sieciach TCP/IP. Określa ona dokładny format wszystkich danych przesyłanych przez sieć.
- ☒ Definicję operacji trasowania, wykonywanej przez oprogramowanie IP, polegającej na wybieraniu trasy, którą będą przesyłane dane.
- ☒ Zawiera zbiór reguł, które służą do realizacji zawodnego przenoszenia pakietów. Reguły te opisują, w jaki sposób węzły i routery powinny przetwarzać pakiety, jak i kiedy powinny być generowane komunikaty o błędach oraz kiedy pakiety mogą być porzucane.

Zadania i właściwości protokołu IP

⌘ Właściwości:

- ☒ IP jest **protokołem bezpołączeniowym**, tzn. nie ustanawia w żaden sposób połączenia i nie sprawdza gotowości odległego komputera do odebrania przesyłanych danych.
- ☒ IP jest **protokołem niepewnym**, tzn. nie zapewnia korekcji i wykrywania błędów transmisji

⌘ Zadania:

- ☒ Definiowanie datagramów, trasowanie (rutowanie) datagramów skierowanych do odległych hostów
- ☒ Definiowanie schematu adresowania używanego w sieci Internet – z identyfikacją hostów, sieci i połączeń między sieciami
- ☒ Dokonywanie fragmentacji i ponownej defragmentacji datagramów IP

Datagram IP



- ⌘ Podstawowa jednostka przesyłanych danych nazywana jest datagramem IP.
- ⌘ Datagram IP podzielony jest na nagłówek i dane.
- ⌘ Datagram IP może lecz nie musi być tylko jednym z wielu fragmentów pakietu IP
- ⌘ Datagram przypomina ramkę sieci fizycznej. Nagłówek ramki zawiera adresy fizyczne, zaś nagłówek datagramu adresy IP.
- ⌘ Ponieważ przetwarzaniem datagramów zajmuje się oprogramowanie, zawartość i format datagramów nie są uwarunkowane sprzętowo

...and the fact that the *Journal of Management Studies* is a leading journal in the field of management studies, it is a great pleasure to have this special issue.

4	8	12	16	20	24	28	32
Wer- sja	IHL	ToS/ DSCP-ECN	Długość całkowita				
Identyfikator			Flagi	Przesunięcie fragmentacji			
Czas życia /TTL		Protokół	Suma kontrolna				
Adres źródłowy							
Adres docelowy							

Budowa datagramu IPv4

(II)



- ⌘ Wersja – [4 bity] – numer wersji protokołu IP. Najpopularniejsza to wersja 4 (standard) oraz nowa wersja 6. Opisana została wersja 4
- ⌘ IHL – [4 bity] – (*Internet Header Length*) jest długością nagłówka w słowach. Minimalna wartość to 5, jednostka: słowa 32 bitowe.
- ⌘ Typ usługi – [8 bitów] – ToS (*Type of Service*) opisuje jakość wymaganej usługi. Zawiera flagi definiujące pierwszeństwo, niezawodność, opóźnienie krytyczne – interpretowane zależnie od warstw wyższych. W najstarszych 3 bitach można umieścić CoS
- ⌘ DSCP/ECN – [8 bitów] – od 1998 zastępuje ToS. 8 bitów podzielono na 6 bitowy *Differential Services Code Point* identyfikujący klasę QoS oraz 2 bitowy kod *Explicit Congestion Notification* obsługujący przypadki przeciążeń ruterów przekazujących ruch

Budowa datagramu IPv4

(III)



- ⌘ DSCP – określa priorytet QoS. Znaczenie wartości częściowo otwarte, jednak z puli 64 możliwości wybrane są 4 równo uprawnione klasy, w której następuje kategoryzacja do 3 grup ważności: Low Drop, Med. Drop, High Drop). W Cisco IOS powyższe 4 klasy mapowane są do wartości parametru *precedence* regulowanej w *service-policy*
- ⌘ ECN – umożliwia routerowi w stanie przeciążenia dodanie do datagramu IP kodu (wartość: 11) informującego odbiorcę, aby użył wobec nadawcy (z którym ma kontakt, gdy stara wartość : 01 lub 10) mechanizmu spowalniającego transmisję (np. *ICMP Source Quench*)
- ⌘ Długość całkowita – [16 bitów] – jest długością pakietu IP w bajtach (zawierającego nagłówek i dane)
- ⌘ Identyfikator – [16 bitów] – wartość identyfikacyjna przypisana nadawanemu pakietowi przed fragmentacją (jeżeli miałaby ona miejsce). W przypadku fragmentacji określa ona przynależność fragmentu do datagramu

Budowa datagramu IPv4 (IV)



- ⌘ Flagi – [3 bity] – flagi sterujące:
 - ☒ bit nr 0: - zarezerwowany, musi mieć wartość zero;
 - ☒ bit nr 1: DF (*Don't Fragment*)- 0 – można fragmentować, 1- nie wolno fragmentować;
 - ☒ bit nr 2: MF (*More Fragments*)- 0 – ostatni fragment, 1 - jest jeszcze więcej. 0 – datagram nie będący fragmentem pakietu
- ⌘ Przesunięcie fragmentacji – [13 bitów] – pole to wskazuje, do którego miejsca pakietu danych należy ten fragment. Przesunięcie fragmentu jest mierzone w jednostkach 8 po bajtów (64 bity). Pierwszy fragment ma przesunięcie równe zero.
- ⌘ Czas życia – [8 bitów] – TTL - pole to wskazuje maksymalny czas przebywania pakietu w Internecie (*Time-to-Live*) wyrażony w „przeskokach” przez bramki.

Budowa datagramu IPv4 (V)



- ⌘ Protokół – [8 bitów] – pole to wskazuje numer protokołu warstwy wyższej, do którego zostaną przekazane dane z tego pakietu.
- ⌘ Suma kontrolna – [16 bitów] – suma kontrolna nagłówka. Ponieważ nagłówki ulegają ciągłym zmianom (np. czas życia) jest ona obliczana i sprawdzana za każdym razem, gdy dany nagłówek jest przetwarzany.
- ⌘ Adres źródła – [32 bity] – adres IP źródła danych.
- ⌘ Adres przeznaczenia – [32 bity] – adres IP komputera docelowego.
- ⌘ Opcje – [długość pola jest zmienna] – mogą zajmować przestrzeń na końcu nagłówka IP.
- ⌘ Uzupełnienie – [długość pola jest zmienna] – jeśli pole opcji nie zajmuje pełnego słowa to zostaje uzupełnione do 32 bitów

Budowa datagramu IPv4 (VI)

- ⌘ Identyfikator protokołu enkapsulowanego - istnieje spójny system rejestracji protokołów warstw wyższych, umożliwiający ustalenie według jakiego protokołu przysłane dane mają być dalej obrabiane. Przykładowo:

⏏ IP	0	Pseudo Protocol Number
⏏ ICMP	1	Internet Control Message Protocol
⏏ IGMP	2	Internet Group Management Protocol
⏏ TCP	6	Transmission Control Protocol
⏏ UDP	17	User Datagram Protocol

- ⌘ Każdy rodzaj sieci ma określony maksymalny rozmiar pakietu MTU (*Maximum Transmission Unit*). W trakcie przekazywania danych, może się okazać, że MTU właściwy dla jednej z sieci, jest zbyt duży dla następnej. Zachodzi wtedy zjawisko fragmentacji pakietu.

Popularne opcje IPv4 (extension headers)



- ⌘ W IPv4 spotkać można następujące opcje (ich pula jest otwarta):
- ⌘ *Security* – pole teoretycznie interpretowane przez niektóre rutery rządowe i informujące o zakazie rutowania datagramu przez obszary nietolerowane. W praktyce opcja ta jest ignorowana
- ⌘ *Strict source routing* – definiuje kompletną ścieżkę od źródła do celu (jako listę adresów IP) – którą pakiet powinien podążać
- ⌘ *Loose source routing* – wymaga, aby pakiet odwiedził listę wyspecyfikowanych adresów IP (po osiągnięciu któregoś z nich jest on usuwany z listy)
- ⌘ *Record route* – każe każdemu ruterowi dodać do załączonej w opcji listy swój adres IP
- ⌘ *Timestamp* – to *Record route* z , lecz dodatkowo z poleceniem zamieszczenia sygnatury czasowej (32 bitowy *timestamp*)

1. *Journal of the American Medical Association*, 1997; 277: 1039-1043.

	4	8	12	16	20	24	28	32
Wer- sja	Differential Services		Etykieta przepływu					
Długość danych			Następny Nagłówek			Liczba przeskoków		
Adres źródłowy (128 bitów)								
Adres docelowy (128 bitów)								

Budowa datagramu IPv6 (II)



- ⌘ Wersja - [4 bity] – analogicznie jak w przypadku IPv4 pole określa wersję protokołu (wartość dla IPv6)
- ⌘ Differential Services – [8 bitów], zawiera analogicznie jak w IPv4 DSCP/ECN: 6-bitowy *Differential Services Code Point* identyfikujący klasę QoS oraz 2 bitowy kod *Explicit Congestion Notification*
- ⌘ Etykieta przepływu (Flow label) – [20 bitów] – umożliwia wprowadzenie pseudo-połączeń, poprzez identyfikowanie datagramów wymagających podobnego traktowania (mających taką samą wartość flow label)
- ⌘ Długość danych – [16 bitów] – w odróżnieniu od IPv4 wartość ta nie bierze pod uwagę długości podstawowego nagłówka (lecz wliczając extension headers)

Budowa datagramu IPv6 (III)

- ⌘ Następny nagłówek [8 bitów] – pole to występuje zarówno w zasadniczym nagłówku IPv6 jak i w każdym extension header. Identyfikuje typ następnego nagłówka. Ostatni nagłówek zawiera w tym polu wartość identyfikatora protokołu enkapsulowanego warstwy wyższej ISO OSI (np. UDP, TCP)
- ⌘ Liczba przeskoków (Hop limit) – [8 bitów] – odpowiednik TTL (czasu życia) w IPv4 - pole mające znaczenie i przetwarzane identycznie jak w IPv4. Nazwę pola zmieniono, aby definitywnie przerwać kojarzenie jego znaczenia z czasem w sekundach (w IPv4 to pole pierwotnie miało utrzymywać czas, ale nie było tak wykorzystywane)
- ⌘ Adres źródłowy - [128 bitów] – adres hosta wysyłającego datagram
- ⌘ Adres docelowy - [128 bitów] – adres hosta będącego adresatem datagramu

Opcje IPv6 (extension headers)

- ⌘ Każda opcja jest kodowana indywidualnie (istnieje wiele formatów danych dla opcji). Każda opcja zawiera pole *następny nagłówek*.
- ⌘ Opcje w IPv6 mogą mieć zarówno stałą długość jak i zawierać dane o zmiennej długości - rejestr wpisów kodowanych jako trójki: typ (1 bajt), długość (1 bajt), wartość (zgodnie z treścią pola długość). Pierwsze dwa bity pola *typ* mówią routerom nie potrafiącym rozpoznać informacji w rejestrze, co zrobić z datagramem: zignorować opcję, usunąć datagram, usunąć datagram i wysłać ICMP, usunąć datagram i wysłać ICMP ale tylko dla ruchu unicast)
- ⌘ Hop-by-hop options – ogólne informacje dla routerów, obecnie główne zastosowanie to wspieranie datagramów >64kB (tzw. *Jumbograms* lub *Jumoframes*). Wówczas wartość opcji to 32 bity określające długość datagramu (standardowe pole nie jest wtedy używane)
- ⌘ Destination options – dodatkowa informacja dla hosta-odbiorcy danego datagramu IP

Opcje IPv6 (extension headers), cd.



- ⌘ Routing – lista ruterów, przez które dany datagram musi przejść. Lista jest redukowana, gdy datagram osiągnął dany ruter
- ⌘ Fragmentation – opcja obsługująca fragmentację analogicznie jak w IPv4 (zawiera numer datagramu, numer fragmentu i bit informujący czy istnieją jeszcze dalsze fragmenty). W IPv6 tylko host źródłowy ma prawo dzielić datagramy – dlatego funkcjonalność z tym związana przeniesiono do opcji.
- ⌘ Authentication – informacje dotyczące identyfikacji nadawcy datagramu IP
- ⌘ Encrypted payload - informacje na temat szyfru stosowanego wobec danych datagramu IP

Dzielenie pakietu IP



- ⌘ Rozmiar pakietu IP (czyli maksymalnie 65535 bajtów, lub 2^{32} w opcji Hop-by-hop w IPv6 pozwalającej na JumboFrames), może przekraczać maksymalny rozmiar pakietów warstwy niższej (np. dla IEEE 802.3: 1518 bajtów)
- ⌘ Jeżeli moduł implementujący IP ma do wysłania dany pakiet, to składa zapytanie w warstwie niższej o wartość MTU (*Maximum Transmission Unit*). Jeśli długość pakietu przekracza MTU to pakiet jest fragmentowany
- ⌘ Pakiet przechodzi przez routery pośredniczące w formie podzielonej
- ⌘ Składanie następuje TYLKO u odbiorcy (*non-transparent fragmentation*) – routery pośrednie nie wykonują tej czynności. Złożony pakiet dociera w takim stanie do odbiorcy.

Składanie pakietów IP



- ⌘ Z każdym otwartym niekompletnym pakietem IP (przekazywanym w formie datagramu) związany jest bufor, tablica znaczników.
- ⌘ Nie jest definiowane opóźnienie maksymalne – ewentualny *timeout* dla usługi określany jest w warstwach wyższych
- ⌘ Na podstawie przesunięcia fragment wstawiany jest w odpowiednie miejsce w buforze (na podstawie pola *Przesunięcie fragmentacji* w datagramie)
- ⌘ Wypełnienie trwa aż do otrzymania kompletu fragmentów (danych z datagramów).
- ⌘ Ostatni fragment pakietu IP, (w rozumieniu - najdalej przesunięty w buforze odbiorczym) posiada wyłączony bit *MF* (*More fragments*)

Congestion control w protokołach Internetu



- ⌘ Zbyt wiele datagramów jednocześnie przekazywanych w sieci lub jej fragmencie (łączy) powoduje opóźnienia, utratę tych datagramów lub inne zaburzenia. Sytuacja taka określana jest terminem *congestion* (tłumaczonym najczęściej jako *obciążenie*).
- ⌘ Skutki przeciążenia są zwykle nagłe (*congestion collapse*, *przeciążenie*) – gdy w konsekwencji zbyt dużego ruchu w sieci dodatkowe problemy sprawiają mechanizmy kontrolne, np. powtarzające transmisje datagramów uznanych za utracone, a faktycznie tylko opóźnionych, czy obsługa kolizji
- ⌘ Ilość przekazanych przez sieć użytecznych pakietów (z wykluczeniem np. błędnych, spóźnionych, powtórzonych) określamy mianem *goodput*

Popularne mechanizmy congestion control



- ⌘ *Network provisioning* – rozbudowa/alokacja infrastruktury sieciowej (także dynamiczna) mająca na celu unikanie przeciążeń
- ⌘ *Traffic-aware routing* – kierowanie ruchu sieciowego innymi trasami, niż zagrożone przeciążeniem
- ⌘ *Admission control* – kontrola ilości danych przekazywanych do sieci (odmowa realizacji dalszych usług, mogących spowodować przeciążenie)
- ⌘ *Traffic throttling* – precyzyjne sterowanie przyprawem datagramów z dopuszczeniem do zagrożonego przeciążeniem łączy tylko bezpiecznej ich ilości (prowadzone według zadanej polityki). Pozostałe datagramy są buforowane.
- ⌘ *Load shedding* – niszczenie datagramów, których przekaz może spowodować przeciążenie (prowadzone według zadanej polityki).

Congestion control – terminologia uzupełniająca



- ⌘ *Burst* – duża ilość danych przekazywana chwilowo przez sieć (najczęściej w ściśle określonej chwili czasowej). *Bursty network* – sieć, w której występują duże skoki tempa przekazu danych
- ⌘ *Congestion avoidance* – mechanizmy gwarantujące, że przeciążenie nigdy nie nastąpi (np. rygorystycznie stosujące *load shedding* lub *admission controll*)
- ⌘ *Choke packet* – datagram wysyłany do nadawcy i polecający mu zmniejszenie prędkości nadawania
- ⌘ *Explicit congestion notification* – mechanizmy eliminujące przeciążenia poprzez wprowadzenie systemu ściśle przestrzeganych zezwoleń dla nadawców do nadawania (alternatywa dla mechanizmów używających *choke packets*)
- ⌘ *Traffic shaping* – techniki dynamicznego regulowania tempa przepływu danych przez sieć oraz kontroli nagłych wzrostów tego tempa (*burst*) – z ich adaptacją do możliwości sieci w danej chwili

Congestion control – polityki i mechanizmy

- ⌘ Klasyczne polityki stosowane w *load shedding*:
 - ☒ *Milk* - nowe jest lepsze niż stare
 - ☒ *Wine* - stare jest lepsze niż nowe
- ⌘ Technika RED (*Random Early Detection*) stosowana w *load shedding*:
gdy ryzyko przeciążenia wzrasta niszczone są (przedwcześnie) losowo wybrane datagramy, co powoduje wykrywanie błędów transmisji przez protokoły warstw wyższych i stymuluje zmniejszenie tempa nadawania
- ⌘ *Resource reservation* – zaawansowane mechanizmy *congestion control* wprowadzają pełne rozliczenie charakterystyki strumieni danych, jakie są przesyłane przez wielo segmentową sieć – z rezerwacją przepustowości sieci dla tych strumieni. Uniemożliwiają one jednocześnie dalsze rezerwowanie przepustowości, gdy zasoby sieci zostały wyczerpane. Przykład protokołu realizującego takie mechanizmy to RSVP (*Resource reSerVation Protocol*) pracującego w kooperacji z sieciami MPLS.

Congestion control – polityki i mechanizmy

- ⌘ Mechanizmy stosowane przy *Admission control* lub *Traffic throttling* :
 - ☒ *Leaky bucket* - odpowiednik logiczny wiadra z dziurą. Dane wypływają z danego węzła w stałym tempie (przez dziurę), lub nie wypływają wcale gdy nie ma nic do nadanie (wiadro puste, czas nadawania zajmują inne transmisje)
 - ☒ *Token Bucket* - odpowiednik logiczny wiadra opróżnianego raz na czas – dane są przekazywane w danej chwili czasowej w ilości nie większej niż pojemność wiadra. Jeśli w tej chwili czasowej dane nie zostały zgromadzone w komplecie – przekazywane jest ich mniej (wolne miejsce zajmują inne transmisje)

Congestion control i QoS – nomenklatura



- ⌘ W kolejkowaniu datagramów stosowana jest kolejność FIFO, zwana tu czasem jako FCFS (*First-Come, First Serve*)
- ⌘ *Tail-drop* – zachowanie polegające na kasowaniu nowo otrzymanego datagramu, gdy kolejka jest pełna
- ⌘ W przypadku wielu kolejek muszą być wdrożone procedury obsługujące ich rywalizację. Najbardziej popularny algorytm to *fair queing* oparty na technice pobierania danych z wielu źródeł o nazwie *round-robin* (warianty *byte-by-byte* lub *packet-by-packet*). Round-robin polega na cyklicznym pobieraniu z kolejek.
- ⌘ Rozbudową algorytmu *fair queing* jest *weighted fair queing*, wprowadzający wagi poszczególnych przepływów (rozliczane *per-byte*)

QoS – dalsze techniki klasyfikacji ruchu



- ⌘ *Expedited forwarding* – technika polegająca na wyróżnieniu części ruchu i przekazaniu go do specjalnej, priorytetowej kolejki. Powstają dwie klasy ruchu: *regular* oraz *expedited* (ruch z pierwszeństwem)
- ⌘ *Assured forwarding* – rozbudowana wersja *expedited forwarding*, zakładająca istnienie czterech klas ruchu (czterech kolejek). W każdej kolejce definiowane są dodatkowo dalsze (najczęściej trzy) klasy priorytetu przekazywanej treści (łączna liczba klas to iloczyn ilości kolejek i liczby możliwych wartości priorytetów). Technika postuluje (w przeciwieństwie do *expedited forwarding*) dołączanie do przekazywanych datagramów pól informujących o przynależności do klasy. Technika jest implementowana między innymi w protokole IP

Datagram IP – kategoryzacja QoS

- Wyróżnia się 12 kategorii QoS określanych mianem *Assured Forwarding* i przeliczanych z klas (1-4), kategorii grup ważności (Low, Med, High), wartości DSCP oraz wartości *precedence* (w przypadku Cisco IOS):

	Class 1 Prec 1	Class 2 Prec 2	Class 3 Prec 3	Class 4 Prec 4
Low Drop	AF11 DSCP =10	AF21 DSCP =18	AF31 DSCP =26	AF41 DSCP =34
Med Drop	AF12 DSCP =12	AF22 DSCP = 20	AF32 DSCP =28	AF42 DSCP =36
High Drop	AF13 DSCP =14	AF23 DSCP =22	AF33 DSCP =30	AF43 DSCP =38

- Dodatkowo wartości binarne w postaci: **XXX**000 DSCP przeliczane są do wartości *precedence* binarnie: **XXX** (dosłowne mapowanie CoS Layer2), np.: wartość 8 (001000) DSCP to wartość 1 (001) *precedence*
- Wartość 0 *DSCP* czyli 0 *precedence* to ruch rutynowy, a wartość 46 (101110) DSCP czyli 5 *precedence* to ruch krytyczny

Kategoryzacja QoS – typowe zachowanie urządzeń



- ⌘ W większości zaawansowanych urządzeń sygnatura DSCP w datagramie IP może być przeliczana na CoS według reguł zdefiniowanych przez administratora. Często też urządzenia warstwy drugiej ISO OSI (operujące standardowo tylko na CoS w ramach Ethernet) mają możliwość ingerowania w datagramy IP w celu dokonania konwersji
- ⌘ Gdy urządzenie (np. przełącznik Ethernet) otrzyma datagram IP z wypełnionym DSCP, używa tzw. mapy DSCP-to-CoS
- ⌘ Gdy otrzyma łączem trunk (IEEE 802.1Q) ramkę Ethernet z wypełnionym CoS, używa innej, osobno zdefiniowanej mapy: QoS-to-DSCP (gdy ramka ma być wysłana do następnego łącza już bez tagowania).
- ⌘ Gdy otrzyma obydwie informacje jednocześnie – korzysta ze swojej konfiguracji określającej, której z nich ufać (DSCP czy CoS).
- ⌘ Gdy musi wygenerować sztucznie CoS (np. wpuszczając nieoznakowaną ramkę do trunk) – przypisuje wartość najniższą (ruch rutynowy).

Kategoryzacja – mapa QoS, DSCP i precedence

DSCP kod	DSCP (bin)	DSCP (dec)	DSCP (hex)	ToS (bin)	ToS (dec)	Precedence (bin)	Precedence (dec)	TOS String Format
none	000000	0	0×00	00000000	0	000	0	Routine
cs1	001000	8	0×08	00100000	32	001	1	Priority
af11	001010	10	0×0A	00101000	40	001	1	Priority
af12	001100	12	0×0C	00110000	48	001	1	Priority
af13	001110	14	0×0E	00111000	56	001	1	Priority
cs2	010000	16	0×10	01000000	64	010	2	Immediate
af21	010010	18	0×12	01001000	72	010	2	Immediate
af22	010100	20	0×14	01010000	80	010	2	Immediate
af23	010110	22	0×16	01011000	88	010	2	Immediate
cs3	011000	24	0×18	01100000	96	011	3	Flash
af31	011010	26	0×1A	01101000	104	011	3	Flash
af32	011100	28	0×1C	01110000	112	011	3	Flash
af33	011110	30	0×1E	01111000	120	011	3	Flash
cs4	100000	32	0×20	10000000	128	100	4	FlashOverride
af41	100010	34	0×22	10001000	136	100	4	FlashOverride
af42	100100	36	0×34	10010000	144	100	4	FlashOverride
af43	100110	38	0×26	10011000	152	100	4	FlashOverride
cs5	101000	40	0×28	10100000	160	101	5	Critical
Ef	101110	46	0×2E	10111000	184	101	5	Critical
cs6	110000	48	0×30	11000000	192	110	6	Internetnetworkcontrol
cs7	111000	56	0×38	11100000	224	111	7	Networkcontrol

Protokół ARP i współpraca z IPv4



- ⌘ Każdy interfejs IP ma przypisany 32-bitowy adres jednoznacznie identyfikujący go w sieci IP, lecz dwie maszyny w sieci Ethernet mogą się komunikować tylko wtedy gdy dodatkowo znają nawzajem swoje adresy MAC
- ⌘ Istnieje potrzeba przekształcenia adresu IP na adres MAC tak, gdyż adres MAC nie jest przypisany na stałe do IP (IP można zmieniać)
- ⌘ Nie ma sposobu na zakodowanie 48-bitowego adresu Ethernet w 32-bitowym adresie IP (zbyt mała długość)
- ⌘ Zapytanie ARP to broadcast w warstwie drugiej. Odpowiedź jest wysyłana już w trybie unicast
- ⌘ Przełącznik LAN i każda stacja DTE archiwizuje w swojej tablicy MAC dane o adresach MAC innych interfejsów i odpowiadających im adresach IP

ARP przy konfigurowaniu adresacji IP



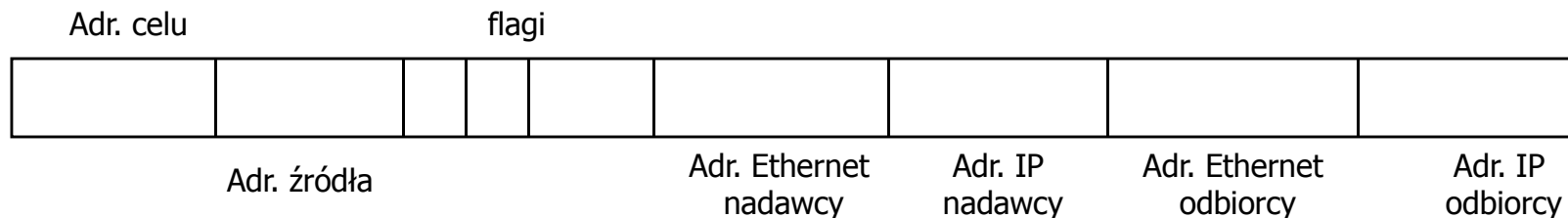
- ⌘ Istnienie protokołu likwiduje konieczność przechowywania tablicy przekształcania adresowego
- ⌘ Przy rozgłaszaniu każda maszyna w segmencie sieci musi przetworzyć pakiet ARP – nawet jeżeli nie weźmie udziału w dalszej komunikacji w ramach tego protokołu
- ⌘ Mogą pojawiać się opóźnienia w konfiguracji sieci po zmianach w interfejsach komunikacyjnych. W celu ich redukcji większość systemów operacyjnych w chwili wykrycia faktu podłączenia do sieci (*link up*) lub zmiany adresu IP przez użytkownika automatycznie wysyła ramkę *broadcast* w celu zidentyfikowania swoich danych w innych stacjach (mechanizm jest wykorzystywany także do szybkiego wykrywania kolizji adresów IP)

ARP - zasada działania (II)



- ⌘ Przypadek gdy host docelowy nie znajduje się w podsieci. Wobec braku odpowiedzi ARP, host pytający ponawia zapytanie wysyłając pakiet na adres gateway'a
- ⌘ Przypadek proxy-arp - ruter symuluje urządzenie, którego dotyczy zapytanie – korzystając ze swojej tablicy routingu
- ⌘ Protokół ARP może być wykorzystywany w celu uniknięcia nadania dwóch takich samych adresów IP w jednej sieci lokalnej. W momencie inicjacji systemu komputer wysyła ramkę ARP z zapytaniem o adres IP, który aktualnie został mu przydzielony
- ⌘ Pakiet Gratuitous ARP Request – jest wysyłany przez urządzenie nowo podłączone do segmentu sieci lub urządzenie, dla którego właśnie zmieniono adres IP. Urządzenie takie informuje wówczas o swoim MAC i IP. Pakiet jest także wykorzystywany do wykrywania kolizji adresacji IP w segmencie sieci – zgłoszenie kolizji następuje poprzez wysłanie Gratuitous ARP Reply z innego (kolidującego) urządzenia.

Format pakietu ARP



- ⌘ Nagłówek pakietu sieci Ethernet
- ⌘ 28 znaków zapytania ARP, z polami częściowo niewypełnionymi
- ⌘ Przetwarzanie zapytania polega na wypełnieniu pustych pól i odesłaniu pakietu

ARP - Odwzorowanie adresów i pamięć podręczna



- ⌘ Pamięć podręczna stosowana jest z uwagi na zbyt duży koszt ciągłego przesyłania ARP
- ⌘ W pamięci podręcznej przechowywane są ostatnio uzyskane powiązania adresu IP z adresem fizycznym
- ⌘ Stacja A wysyłając prośbę o adres fizyczny stacji B od razu dowiazuje informację o swoim adresie fizycznym
- ⌘ Jeśli w stacji zostanie zmieniony adres fizyczny (np. w wyniku zmiany karty sieciowej), to może on bez zapytania o jego adres fizyczny rozgłosić go do innych stacji, tak aby uaktualniły informacje w swoich pamięciach podręcznych
- ⌘ Wariant ARP Stuffing (No ARP) - jednorazowe skonfigurowanie adresu IP i MAC urządzenia mobilnego ze zdalnej stacji - poprzez nawiązanie z niej pierwszej komunikacji (np. ICMP/ping) z urządzeniem mobilnym - używając wspomnianych par adresów.

ARP, InverseARP i RARP



- ⌘ ARP (Address Resolution Protocol) - Protokół odwzorowania adresów - umożliwia uzyskanie adresu MAC innej stacji na bazie jej adresu IP
- ⌘ InverseARP (Inverse Address Resolution Protocol) - protokół odwrotnego odwzorowania adresów, umożliwia uzyskanie adresu IP innej stacji na bazie jej adresu MAC
- ⌘ RARP, ReverseARP (Reverse Address Resolution Protocol) - służy do konfigurowania adresu IP własnej stacji na bazie adresu MAC (zastąpiony przez BOOTP, potem przez DHCP)
- ⌘ ARP spoofing - atak bazujący na fałszowaniu odpowiedzi ARP
- ⌘ Proxy ARP - wykorzystanie ARP do przekierowania odpowiedniego ruchu do wybranej stacji (np. w celu tunelowania)

NDP (Neighbor Discovery Protocol) dla IPv6

- ⌘ Odpowiednikiem ARP dla wsparcia procesu propagacji wiedzy odpowiednikach MAC i IPv6 jest protokół NDP. Operuje on w warstwie drugiej ISO OSI, ale korzysta z datagramów ICMPv6
- ⌘ Umożliwia identyfikację adresów MAC hostów zdalnych, wykrywanie kolizji adresów IP, identyfikację ruterów (*router solicitation*), innych hostów (*neighbor solicitation*), identyfikację prefiksów sieci dla IPv6, poszukiwanie serwerów DNS, ciągła kontrolę osiągalności wcześniej rozpoznanych węzłów sieci itp.
- ⌘ 5 istotnych dla NDP komunikatów ICMPv6:
 - ⏏ Router Solicitation, kod 133 i Neighbor Solicitation, kod 135
zapytanie odpowiednio o obecność ruterów mogących być bramkami z lokalnego segmentu sieci lub innych hostów
 - ⏏ Router Advertisement, kod 134 i Neighbor Advertisement, kod 136
odpowiedź rutera/hosta na to zapytanie
 - ⏏ Redirect, kod 137, ruter odpowiada wskazując inny ruter-bramkę

NDP (Neighbor Discovery Protocol) dla IPv6

⌘ Rozszerzenia NDP:

- ⏏ Inverse Neighbor Discovery (IND) służy do rozgłaszania wiedzy o adresach IP powiązanych z odpowiednimi adresami warstwy drugiej ISO-OSI
- ⏏ *Secure Neighbor Discovery Protocol* (SEND) – wersja zabezpieczona, stosowana głównie na użytek IPsec
- ⏏ Neighbor Discovery Proxy (ND Proxy) – odpowiednik proxy ARP, umożliwiający mostkowanie segmentów sieci posługujących się adresacją IPv6 w jeden segment sieci IP
- ⏏ Neighbor unreachability detection (NUD) – oparta o timery funkcjonalność pozwalająca na ciągłą kontrolę obecności wcześniej aktywnych hostów (przy pomocy Neighbor Solicitation/Neighbor Advertisement)

⌘ NDP wymaga skonfigurowania w urządzeniach pełniących funkcje ruterów (bramkek)

Protokół ICMP



- ⌘ ICMP (*Internet Control Message Protocol*)
- ⌘ Jest częścią warstwy czwartej OSI
- ⌘ Do swojego transportu wykorzystuje datagramy IP
- ⌘ Obsługuje większość sytuacji awaryjnych i informuje o nich zainteresowane hosty
- ⌘ Wykorzystywany do diagnostyki, między innymi przez komendy ping i traceroute

Przykłady stosowania ICMP (I)

⌘ Wykrywanie nieosiągalnych miejsc przeznaczenia – jeśli komputer docelowy nie odpowiada to system, który wykrył problem wysyła do nadawcy komunikat *Destination Unreachable*:

☒ Jeśli komunikat ten jest wysyłany przez ruter, oznacza, że ruter nie może wysyłać pakietów do danego komputera. Może to nastąpić w dwóch przypadkach:

- adres docelowy IP nie istnieje - *Host-unreachable*,
- ruter nie może dostarczyć datagramu do tej sieci - *Network-unreachable*.

☒ W momencie, gdy komunikat ten jest wysyłany przez host, może to oznaczać, że:

- dany komputer nie posiada wsparcia dla któregoś z protokołów - *Protocol-unreachable*,
- port TCP/UDP jest nieosiągalny- *Port-unreachable*.

Przykłady stosowania ICMP (II)



- ⌘ Sterowanie przepływem danych – w przypadku, gdy komputer docelowy transmisji IP nie nadażą za obróbką przychodzących datagramów IP, ICMP wysyła komunikat *Source Quench*, po którym nadawca czasowo wstrzymuje transmisję.
- ⌘ Przekierowywanie ścieżek – jeśli komputer, do którego dotarł datagram IP uzna, że właściwszą bramką będzie inny komputer z tej samej sieci, wysyła komunikat *Redirect* wskazujący na ten właśnie komputer (musi znajdować się w tej samej sieci IP). Po otrzymaniu takiego komunikatu nadawca aktualizuje swoją tablicę routingu

Przykłady stosowania ICMP (III)



- ⌘ Sprawdzanie obecności zdalnego hosta – wysyłany jest komunikat *Echo Message*, po otrzymaniu którego komputer docelowy musi odpowiedzieć. Jeśli tego nie zrobi, uznawany jest za nieosiągalny. odbywa się podczas wywołania komendy ping
- ⌘ Jeśli datagram IP, podczas przechodzenia przez ruter osiągnie zerowy limit „czasu życia” (*Time-to-Live*) - jest usuwany. Do komputera źródłowego danego datagramu wysyłany jest komunikat ICMP *Time-exceeded*.

Format komunikatu zwrotnego ICMP



- ⌘ Typ - typ (dla komunikatów zwrotnych zawsze równy 3)
- ⌘ Kod operacji – dodatkowy kod zwrotny (gdy potrzebny)

MPLS - podstawy



- ⌘ MPLS – *MultiProtocol Label Switching*
- ⌘ Operuje w warstwach 2 i 3, często zwany warstwą 2.5
- ⌘ Pierwotnie (1997) stworzony dla wprowadzenia uniwersalnych mechanizmów analogicznych do ATM (MPOA - *MultiProtocol Over ATM*) i FDDI, z czasem rozszerzony na inne protokoły
- ⌘ Bazuje na dokładaniu etykiet pomiędzy warstwę łącza danych a sieci i oparciu przełączania na treści tych etykiet
- ⌘ Ścieżki tworzone na trasie datagramów operują na różnych etykietach (często wielu, utrzymywanych na stosie). W konsekwencji przełączania stos może być modyfikowany

MPLS



- ⌘ Definiowana jest tzw. domena (chmura) MPLS – zbiór ruterów pracujących w ramach MPLS oraz łączących je segmentów sieci
- ⌘ Rozwiązuje problem integracji najlepszych cech tradycyjnych technologii warstwy 2 i warstwy 3 – (angażuje protokoły warstwy 2 ISO OSI, a nad nimi nadbudowuje uniwersalną adresację dla sieci wielo segmentowej)
- ⌘ Analogie etykiet do VPI/VCI używanych w ATM czy DLCI we Frame Relay
- ⌘ Podobnie – analogie pomiędzy ścieżkami LSP (o nich później) do PVC/SVC w ATM czy informacji o przełączaniu Frame Relay

MPLS - adresacja

- ⌘ FEC – *Forwarding Equivalence Class* – kategoria datagramów IP posiadających wspólne cechy – najczęściej wspólny cel trasowania po ścieżce MPLS (LDP – Label Distribution Path). Etykieta MPLS w datagramie służy między innymi do zidentyfikowania FEC datagramu (poprzez powiązania – *bindings*). Często stosowaną techniką interpretacji rozróżnienia FEC w ruterach wyprowadzających ruch z chmury MPLS jest wiązanie ich z ruterami wirtualnymi w technologii VRF (*Virtual Routing and Forwarding*) za pośrednictwem definiowanych tam identyfikatorów o nazwie *Route Distinguisher* (RD). Format identyfikatora: **IDSystemuAutonomicznego:IDRuteraLokalnego**
- ⌘ Stos MPLS – struktura LIFO. Datagram bez etykiety posiada stos o głębokości 0, na dole stosu znajduje się „poziom 1”, nad nim kolejne – wyższe. Każdy ruter MPLS może kłaść na stos kolejne etykiety – skutkujące później wyborem ścieżki trasowania datagramu. Przekazanie w routerze MPLS datagramu dalej bazuje głównie na rozpoznaniu wartości etykiety znajdującej się na szczycie stosu.

MPLS – klasyfikacja ruterów



- ⌘ Ruter MPLS z funkcjonalnością P – (*Provider*) – ruter w szkielecie chmury MPLS
- ⌘ Ruter MPLS z funkcjonalnością PE – (*Provider Edge*) – ruter wyprowadzający ruch do tzw. CE – *Customer Equipment*
- ⌘ LSR - *Label Switching Router* - odnosi się do każdego rutera w chmurze MPLS, lecz głównie do takiego, który nie wyprowadza ruchu MPLS poza chmurę (ruter P)
- ⌘ LER - *Label Edge Router* – ruter, który wyprowadza ruch MPLS poza chmurę (ruter PE)
- ⌘ *Upstream Router* dla datagramów – ten, który wysyła datagramy MPLS
- ⌘ *Downstream Router* (*Router D*) dla datagramów – ten który odbiera datagramy MPLS

MPLS NHLFE

- ⌘ NHLFE - *Next Hop Label Forwarding Entry* – definicja, mówiąca o tym co robić przy natrafieniu na daną ramkę. Operuje na tzw. tokenach.
- ⌘ Translacja prowadzona na podstawie treści NHLFE:
Token-> akcja na stosie etykiet + nowa etykieta na wierzchołku stosu + określenie interfejsu wyjściowego i następnej bramki IP
- ⌘ W konsekwencji następuje określenie:
 - ⏏ Po pierwsze – dokąd przełączyć datagram
 - ⏏ Po drugie - jakie operacje na stosie MPLS datagramu wykonać:
 - ⏏ *Replace* – zamian etykiety z wierzchołka stosu na inną
 - ⏏ *Pop* – zdjęcie etykiety ze stosu
 - ⏏ *Replace and push* - zamiana (jak w pierwszym przypadku) lecz także dodanie na stos serii innych etykiet
- ⌘ NHLFE może także określić tryb enkapsulacji warstwy łącza danych w następnym LSP.

MPLS ILM



- ⌘ ILM - *Incoming Label Map* odwzorowuje etykietę przychodzącego datagramu do zbioru NHLFE (umożliwia przydzielanie wartości tokenów datagramom)
- ⌘ Translacja na podstawie treści ILM:
Interfejs wejściowy + etykieta -> wartość tokenu dla datagramu
- ⌘ Jest konieczna dla rozpatrzenia każdego datagramu, który wchodzi do rutera posiadając już etykietę MPLS
- ⌘ Jeśli ILM odwzorowuje konkretną etykietę MPLS do zbioru więcej niż jednej NHLFE – to dwuznaczność można wykorzystać przy wprowadzaniu *load ballancing* (alternatywne ścieżki)
- ⌘ W konsekwencji - ILM określa tak zwaną LSP – *Label Switching Path* – sekwencja ruterów, przez które datagram musi przejść aby dojść do celu.

MPLS FTN



- ⌘ FTN - *FEC-to-NHLFE*
- ⌘ Tablica, która odwzorowuje FEC do zbioru NHLFE
- ⌘ Uzupełnienie ILM, która odwzorowywała etykietę przychodzącego datagramu do zbioru NHLFE
- ⌘ Konieczna wtedy, gdy przekazywane pakiety przychodzą jeszcze bez etykiet, ale mają mieć nadaną etykietę MPLS zanim zostaną wysłane dalej
- ⌘ Ponownie (w przypadku dwuznaczności) można aplikować tu *load ballancing*
- ⌘ Translacja na podstawie treści FTN:
FEC (zakres adresów IP) -> wartość tokenu dla datagramu

MPLS – wprowadzanie datagramów do chmury



- ⌘ Pakiet bez etykiety odbierany jest z LER przez LSR
- ⌘ W celu przekazania pakietu bez etykiety MPLS, ruter LSR analizuje nagłówki IP datagramu i interfejs (w tym także VRF, czyli w konsekwencji VLAN, przez który datagram przyszedł)
- ⌘ Na tej podstawie określa FEC dla datagramu (biorąc tu pod uwagę np. *Route Distinguisher*)
- ⌘ Stosuje FTN by odwzorować FEC w NHLFE
- ⌘ Na podstawie instrukcji z NHLFE określa gdzie przekazać datagram. Wykonuje też operację na stosie etykiet MPLS pakiet
- ⌘ Generuje ramkę i wysyła ją do następnego rutera w chmurze MPLS (*Downstream router*)

MPLS LDP



- ⌘ LDP - *Label Distribution Protocol*
- ⌘ Zbiorów procedur, dzięki którym jeden ruter LSR informuje inne routery o powiązaniach:
etykieta MPLS <-> FEC
- ⌘ Cel nadrzędny: automatyczne przekazywanie informacji o klasyfikacji FEC i regułach postępowania z klasyfikowanymi datagramami
- ⌘ Jeżeli dwa routery wymieniają informacje w ramach LDP to określamy je mianem *label distribution peers*
- ⌘ Protokół ma wersje wyspecjalizowane (uwaga na kompatybilność przy wyborze protokołu!), np. MPLS-BGP, MPLS-RSVP-TUNNELS
- ⌘ Zakłada się, że w danej chmurze MPLS istnieje tylko jeden wariant protokołu LDP

MPLS LDP



- ⌘ Głównie zadanie protokołu: kojarzenie *Forwarding Equivalence Class* (FEC) z każdą stworzoną LSP (*Label Switched Path*) oraz tworzenie i utrzymywanie LSP – *Label Switched Paths*
- ⌘ LDP jest dwukierunkowy – dane o trasach wymieniane są w obydwu kierunkach, pomiędzy LSR peers
- ⌘ LDP nie wymaga obecności konkretnego protokołu routingu w każdym skoku (hop) wzdłuż ścieżki LSP (*Label Switched Path*) by zestawić tą LSP (ruch LDP może iść inną drogą tą, którą niż pokonają w przyszłości datagramy z etykietami)
- ⌘ Możliwe jest także stosowanie jawnie rutowanych LSP (*Explicit Routed LSP*) – czyli konfigurowanie ich zgodnie z narzuconymi regułami Inżynierii Ruchu (TE - *Traffic Engineering*)

MPLS LDP – klasy wiadomości



⌘ Cztery klasy wiadomości:

- ☒ *Discovery* - utrzymanie obecności rutera LSR w sieci (wysyłanie *LDP Hello* na port UDP na adres multicast lub *LDP Targeted Hello* na port UDP konkretnego *LSR peer*)
- ☒ *Session* - zestawienie, utrzymywanie i zakończenie sesji LDP między *LDP peers*
- ☒ *Advertisement* - tworzenie, zmiana i kasowanie odwzorowania etykiety dla FEC
- ☒ *Notification* – rutynowe dostarczanie informacji doradczych i informowania o błędach

MPLS LDP – zarządzanie etykietami



⌘ Trzy parametry:

- ⏏ Technika czynnego przekazywania informacji o etykietach pomiędzy ruterami:
 - ⏏ *downstream iunsolicited (DU)*
 - ⏏ *downstream-on-demand (DoD)*

- ⏏ Technika sterowania ścieżkami LSP w przeciwnym kierunku:
 - ⏏ *ordered*
 - ⏏ *independent*

- ⏏ Technika podtrzymywania informacji o etykietach:
 - ⏏ *liberal*
 - ⏏ *conservative*

MPLS LDP, parametry

⌘ Sposób przekazywania informacji o etykietach:

- ☒ *Downstream Unsolicited* (DU, tryb bez zapowiedzi) - wysyłanie przyporządkowania etykiety do FEC do rutera *upstream* (czyli tego, który wysyła datagramy MPLS do rutera aktualnie rozpatrywanego) bez wcześniejszego żądania od niego. Przebiega automatycznie – trasa jest budowana samoistnie mimo braku zapotrzebowania na nią
- ☒ *Downstream-on-Demand* (DoD, tryb na żądanie) - przyporządkowania etykiety do FEC wysyłane tylko w odpowiedzi na jawne żądanie rutera *upstream*

MPLS LDP, parametry

⌘ *Label Distribution Control Mode*

- ☒ *Independent LSP Control (niezależna kontrola)* - etykieta upstream może zostać rozgłoszona zanim zostanie otrzymana etykieta downstream, gdyż każdy LSR może rozgłaszać odwzorowanie etykiet do swoich sąsiadów w dowolnym czasie
- ☒ *Ordered LSP control (tryb uporządkowany kontroli)* - LSR może odwzorowywać etykiety tylko dla FEC dla których ma przydzielone odwzorowanie etykiet dla następnego skoku FEC lub dla którego LSR jest ruterem wyjściowym z MPLS (*egress router*)

MPLS LDP, parametry

⌘ *Label Retention Mode - Liberal:*

- ☒ Tryb Liberal umożliwia szybkie reagowanie na zmiany lecz powoduje przetrzymywanie niepotrzebnych etykiet.
- ☒ W trybie DoD (*Downstream on Demand*) LSR żąda odwzorowania etykiet dla wszystkich znanych prefiksów FEC od wszystkich LSR peer.
- ☒ W trybie DU (*Downstream Unsolicited*) każde odwzorowanie etykiety otrzymane od LSR peer jest zachowywane, niezależnie od tego, czy ten LSR jest następnym skokiem na ścieżce dla rozgłoszonego odwzorowania

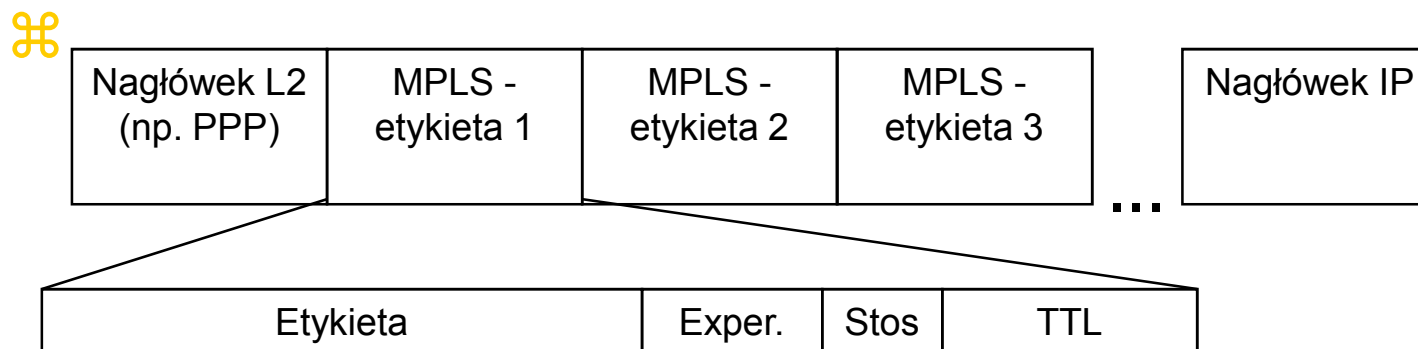
MPLS LDP, parametry

⌘ *Label Retention Mode – Conservative:*

- ☒ W trybie DoD (*Downstream on Demand*) LSR będzie żądał odwzorowań etykiet tylko od rutera LSR następnego skoku (z punktu widzenia trasy rutowania)
- ☒ W trybie rozgłaszania DU (*Downstream Unsolicited*) rozgłoszone odwzorowania etykiet są zachowywane tylko jeśli będą wykorzystane do przekazywania pakietów

MPLS – format etykiety

- ⌘ 32 bity (nie dotyczy sieci ATM i Frame Relay), w tym:
 - ⏏ 20-bitowy identyfikator etykiety MPLS
 - ⏏ 3-bitowe pole eksperymentalne (używane jako CoS kompatybilne z QoS)
 - ⏏ 1-bitowy flaga stanu stosu MPLS
 - ⏏ 8-bitowy znacznik TTL (Time To Live – czasu życia o znaczeniu analogicznym do stosowanego np. w datagramach IP)



MPLS format etykiety w przypadku ATM i Frame Relay

- ⌘ W przypadku budowania sieci MPLS nad ATM etykieta jest przechowywana w VPI i VCI celki ATM.
- ⌘ Stosuje się jeden z trzech wariantów kodowania (dla sygnalizacji ATM NNI):
 - ☒ Pierwsza etykieta MPLS (fragment) jest przechowywana w polu VPI celki ATM (dla NNI jest to 12 bitów), druga w VCI (16 bitów)
 - ☒ Istnieje tylko jedna etykieta, której treść jest podzielona pomiędzy VPI i VCI
 - ☒ Etykieta zajmuje tylko VCI, zaś VPI jest przeznaczony na identyfikator tunelu VPN MPLS

Cisco IOS – konfigurowanie MPLS/LDP

- ⌘ Aktywowanie ramkowania (uwaga: datagramy wychodzące do rutera PE jako downstream nie są już etykietowane – nie ma takiej potrzeby):

Router(config)#mpls ldp router-id Loopback1

Router(config)#interface f0/0

Router(config-if)#ip address 200.200.200.1 255.255.255.0

Router(config-if)#mpls ip

w starszych IOS: Router(config-if)# tag-switching ip

- ⌘ Diagnostyka:

Router(config)#sh mpls forwarding-table

Router(config)#sh mpls ldp bindings

Router(config)#sh mpls ip bindings

- ⌘ Minimalna konfiguracja: dwa routery MPLS PE i jeden ruter MPLS P pomiędzy nimi, dodatkowo źródło i cel transmisji poza chmurą MPLS (np. interfejsy loopback routerów MPLS PE)

VRF – Virtual Routing and Forwarding



- ⌘ Technologia umożliwiająca utrzymywanie wielu równoległych systemów routowania (VRF) w jednym fizycznym urządzeniu routera (każdy z nich posiada własną *forwarding table*) – więc adresy IP hostów pomiędzy poszczególnymi VRF mogą się powtarzać!
- ⌘ Systemy te identyfikujemy poprzez nazwy oraz tzw. *Route-Distinguisher* (RD) – w formacie AdresIP:liczba lub IdentyfikatorAS:liczba
- ⌘ Interfejsy IP routera (zarówno fizyczne jak i emulowane – np. wirtualne lub pod-interfejsy) są dzielone pomiędzy poszczególne VRF
- ⌘ Technologia została opracowana aby umożliwić wyprowadzanie ruchu z MPLS w różnych kierunkach i w tym samym urządzeniu
- ⌘ VRF może jednak także funkcjonować bez MPLS – w przypadku systemów Cisco nosi wtedy nazwę VRF Lite

Cisco IOS – konfigurowanie VRF Lite



- ⌘ Tworzenie systemów VRF:

```
Router(config)#ip vrf SIECI1  
Router(config-vrf)#rd 1111:1111  
Router(config)#ip vrf SIECI2  
Router(config-vrf)#rd 2222:2222
```

- ⌘ Przypisanie interfejsów do VRF:

```
Router(config)#int fa0/1  
Router(config-if)#ip vrf forwarding SIECI1
```

- ⌘ Przypisanie pod-interfejsów do VRF (teraz do interfejsu Fa 0/0 można podłączyć trunk przełącznika Ethernet i rozprowadzić poszczególne VRF poprzez zdefiniowane VLAN do zwykłych ruterów IP):

```
Router(config)#int fa0/0.2  
Router(config-subif)#encapsulation dot1q  
Router(config-subif)#ip vrf forwarding SIECI2  
Router(config-subif)#ip address 5.5.5.5 255.255.255.252
```

Cisco IOS – rutowanie w ramach VRF Lite

- ⌘ Konfigurowanie protokołu routowania dynamicznego do współpracy z wybranym VRF, przykłady:

```
Router(config)#router ospf 2 vrf SIECI2
```

```
Router(config-router)# network 5.5.5.5 0.0.0.255 area 0
```

```
Router(config)# router eigrp 100
```

```
Router(config-router)# address-family ipv4  
vrf SIECI2 autonomous-system 100
```

- ⌘ Diagnostyka:

```
Router#ping vrf SIECI2 5.5.5.5
```

```
Router#show ip route vrf SIECI2
```

```
Router#traceroute vrf SIECI2 5.5.5.6
```

```
Router#show ip vrf interfaces
```

```
Router#show ip arp vrf SIECI2
```

```
Router#show ip protocols vrf SIECI2
```

MPLS VPN



- ⌘ Najbardziej popularna metoda: BGP/MPLS IP VPN (użycie BGP go zarządzania połączeniami VPN)
- ⌘ Zasada działania: ruter MPLS PE (czyli LRE) obsługuje IP VRF, czyli posiada kilka tablic rutowania. Domyślnie jest przeznaczony do obsługi kilku klientów (instytucji) i ich sieci, gdzie klienci ci mają inne sieci dostępne przez chmurę MPLS
- ⌘ Ruch musi być izolowany, dodatkowo mogą wystąpić kolizje adresacji sieci różnych klientów – więc proces rutowania także trzeba segmentować
- ⌘ Nowy rodzaj funkcjonalności rutera – CE (Customer Edge) – pełniący rolę celu dla tunelu MPLS VPN (często fizycznie łączony z PE)
- ⌘ Nazwa robocza technologii MPLS VPN: VPN-IPv4
- ⌘ Adresy: 12 bajtów podzielone na: 8-bajtowy Route Distinguisher (używany w VRF) i 4-bajtowy klasyczny adres IPv4

BGP/MPLS VPN

zasada działania



- ⌘ Gdy ruter PE pozna klasyczną trasę IPv4 od rutera CE (*Customer Edge*) to konwertuje tą trasę na unikatową trasę VPN i eksportuje do BGP
- ⌘ BGP rozgłasza tą trasę do innych ruterów PE, które obsługują przyszłe przeciwległe końce tuneli MPLS VPN
- ⌘ Gdy trasa VPN zostanie wybrana przez proces decyzyjny BGP, jest ona konwertowana z powrotem na postaci trasy IPv4 i importowana do właściwej tabeli VRF w odległym routerze PE.
- ⌘ Trasa z VRF w zdalnym PE jest rozgłaszana do przyłączonych do niego ruterów CE
- ⌘ Dodatkowo: gdy ruter PE rozgłasza trasy VPN poprzez BGP do innych ruterów PE - także przypisuje do niej i rozgłasza etykietę MPLS (nazywaną etykietą VPN)

MPLS VPN

– wybór tras dla datagramu



- ⌘ PE otrzymuje datagram IP od urządzenia (lub procesu) CE i wybiera odpowiednią tablicę VRF (pobiera adres IP przeznaczenia).
Skojarzenie PE-CE-VRF powoduje iż dzieje się to w zależności od wejściowego interfejsu pakietu (numeru VLAN)
- ⌘ Pakiet przechodzi przez chmurę MPLS dostawcy do przeciwnego PE, który jest następnym skokiem BGP – czyli jest formowany do postaci pakietu MPLS z etykietą MPLS VPN dodaną do stosu etykiet (jest tunelowany do rutera następnego skoku BGP)
- ⌘ Zdalny PE określa na podstawie etykiety MPLS VPN instancję VRF – ustalając cel (*next hop* lub interfejs do właściwego urządzenia CE)
- ⌘ Pozbawiony etykiet datagram jest przekazywany do przeciwnego VLAN – co kończy cały proces

MPLS VPN

– właściwości zalety



- ⌘ Możliwe jest budowanie globalnych sieci rutujących pomiędzy VLAN
- ⌘ VPN IPv4 umożliwia utrzymywanie wszystkich tras VPN poza ruterami LSR MPLS nie będącymi ruterami PE (*Provider Edge*) - te routery nie muszą znać tras do urządzeń CE, a jedynie do PE
- ⌘ BGP/MPLS VPN umożliwia dostawcy usług dystrybucję informacji (poprzez BGP) o osiągalności adresów IP pomiędzy rozproszonymi oddziałami instytucji, należącymi do tej samej sieci VPN

Cisco IOS – Tunele MPLS VPN (I)

- ⌘ Tworzenie tunelu MPLS VPN na bazie ścieżki explicite:

```
Router(config)#interface Tunnel1
Router(config-if)#ip unnumbered Loopback0
Router(config-if)#tunnel destination 200.200.206.1
Router(config-if)#tunnel mode mpls traffic-eng
Router(config-if)#tunnel mpls traffic-eng autoroute announce
Router(config-if)#tunnel mpls traffic-eng path-option 1
explicit name SCIEZKA1
```

- ⌘ Utworzenie ścieżki (analogicznie w przeciwnym ruterze LER):

```
Router(config)#ip explicit-path name SCIEZKA1 enable
Router(config-path)#next-address 200.200.201.2
Router(config-path)#next-address 200.200.206.1
```

Cisco IOS – Tunele MPLS VPN (II)



- ⌘ Konfigurowanie interfejsu fizycznego dla MPLS (pierwszego na ścieżce):

```
Router(config)#interface FastEthernet0/0
Router(config-if)#ip address 200.200.201.1 255.255.255.0
Router(config-if)# mpls ip
Router(config-if)#mpls traffic-eng tunnels
Router(config-if)#ip rsvp bandwidth 256 256
Router(config-if)#ip load-sharing per-packet
```

- ⌘ Konfigurowanie MPLS LDP:

```
Router(config)# mpls traffic-eng tunnels
Router(config)# mpls ldp router-id Loopback0
```

MPLS RSVP



- ⌘ RSVP (*Resource Reservation Protocol*) dla obsługi i realizacji jawnej ścieżki LSP
- ⌘ *Constraint-based routing* (CSPF) w sieciach IP – umożliwia wywieranie wpływu na zestawianie jawnych ścieżek LSP i rezerwowanie pasma
- ⌘ Komunikaty:
 - ☒ *RSVP path message* - żądanie zestawienia wysłane od LSR do przeciwnego LER
 - ☒ *RSVP reserved message* – potwierdzenie wysłane z przeciwnego LER
 - ☒ *Mapping* – potwierdzenie przesłane do zamawiającego LSR od jego ościennego LSR

Mobile IP



- ⌘ Przeźroczysta migracja hosta - użytkownik ruchomy wraz ze swoim urządzeniem (*mobile host*) przechodzi z jednego segmentu sieci do innego i jest to niewidoczne dla aplikacji (w warstwach wyższych)
- ⌘ Wyróżniani są tzw. *home agent* oraz *foreign agent*, które współpracują ze sobą, aby zapewnić jednostce *mobile host* przemieszczanie bez zmiany adresu IP. *Home agent* zawiera interfejs należący do sieci macierzystej *mobile host*.
- ⌘ Kiedy *mobile host* pojawia się innym niż macierzysty segment sieci, tamtejszy *foreign agent* wykonuje na jego rzecz usługę *proxy ARP* oraz zgłasza do stosownego *home agent* o przybycie. *Foreign agent* jest zatem pośrednikiem rejestracyjnym w obcym segmentie sieci i pośredniczy w komunikacji na linii *Mobile host* - *Home agent* po rejestracji (tunelowanie z enkapsulacją pakietów w innym protokole). Sama detekcja agentów następuje za pośrednictwem ICMP (komunikat *ICMP Router Discovery*).

Mobile IP – funkcjonowanie komunikacji IP

- ⌘ Tworzony jest tunel (GRE) pomiędzy *home agent* i *foreign agent*
- ⌘ Gdy mobile host jest w sieci zdalnej (zarządzanej przez *foreign agent*)
 - datagramy IP adresowane do *mobile host* z sieci Internet (zawierając docelowy/odbiorcy adres IP pasujący do sieci *home agent*) – trafiają właśnie do *home agent* (ten ruter jest bramką dla datagramów). Następnie są przekazywane przez tunel pomiędzy *home agent* i *foreign agent* do sieci zawierającej obecnie *mobile-host*. Finalnie – trafiają do mobile host.
- ⌘ Datagramy wychodzące od *mobile host* do sieci Internet nie muszą pokonywać takiej drogi – są przesyłane od razu do sieci Internet, lecz zawierają adres źródłowy/nadawcy wskazujący na mobile host (nie należący do sieci zdalnej, z której faktycznie datagram IP wysłano)
- ⌘ Problemy Mobile-IP: wymagana jest modyfikacja stosu TCP/IP, mobile-IP koliduje z NAT i systemami IPS

Konfigurowanie mobile IP



- ⌘ Konieczne jest skonfigurowanie funkcjonalności *foreign agent* i *home agent*
- ⌘ Home agent otrzymuje od mobile host tzw. *care-of address*, który jest globalnym adresem mobile host. wskazuje na emulowaną sieć wirtualną (lecz adresowalną z Internetu) zawierającą adresy wszystkich przypisanych *mobile host*
- ⌘ Usługi *home agent* najczęściej rozszerzane są o DHCP i system uwierzytelniania hostów mobilnych

Adresowanie usług sieciowych

- ⌘ Gdy aplikacja potrzebuje połączenia z usługą na zdalnym hoście do uzupełnienia identyfikacji adresowej tej usługi (poza adresem hosta) wykorzystywany jest tzw. port (identyfikowany numerem). Ogólna nazwa specyfikująca adres sieciowej usługi zdalnej to tzw. TSAP (*Transport Service Access Point*). Analogicznie do TSAP w warstwie sieci identyfikują się adres hosta jako NSAP (*Network Service Access Point*). W ramach tego samego NSAP może naturalnie istnieć wiele TSAP
- ⌘ Gdy adresy TSAP nie są ogólnie znane (numer portu jest zmienny) - wykorzystywany jest tzw. *portmapper* (proces poszukujący adresu TSAP adekwatnego dla odpowiedniej usługi nazwanej).
- ⌘ W niektórych sytuacjach adres TSAP może być ustanawiany na żądanie (np.. gdy usługa jest dedykowana dla konkretnego odbiorcy i funkcjonuje tylko chwilowo) – wówczas stosuje się *Initial connection protocol* – pozwalający na negocjowane uruchomienie usługi i poinformowanie odbiorcy o TSAP

Protokół sterowania transmisją (TCP)



- ⌘ *TCP - Transmission Control Protocol*
- ⌘ Zapewnia połączeniową transmisję danych pomiędzy dwoma lub więcej hostami
- ⌘ Formatowane są strumienie danych, których fragmenty przesyłane są w datagramach IP (zawierających datagramy TCP). Dzięki wprowadzeniu identyfikacji liczbowej strumieni (portów TCP) - host może obsługiwać wiele strumieni danych jednocześnie
- ⌘ Brak ograniczeń czasowych dla transmisji strumienia (te, jeśli są, będą narzucane wyłącznie przez aplikacje operujące w warstwach wyższych)
- ⌘ Zawansowane Buforowanie danych w możliwością sterowania przepływem (wypychanie wymuszone (*Flush*) np. przy telnet, optymalizacja przepływu itp..)

TCP - kontrola przepływu i weryfikacja pakietów



- ⌘ Kontrolowanie treści otrzymanego datagramu poprzez sumy kontrolne (CRC)
- ⌘ Technika (metoda) „potwierdzania pozytywnego” z automatyczną retransmisją danych:
 - ☒ Potwierdzenie przyjęcia kolejnej porcji danych wysyłane jest periodycznie do nadawcy
 - ☒ Timeout nadania i automatyczna retransmisja w przypadku nieotrzymania potwierdzenia od odbiorcy
- ⌘ 32 bitowe numerowanie danych (numery sekwencji) – rosnące z liczbą przesyłanych bajtów
- ⌘ Sterowanie przepływem (*Flow control*) - powstrzymywanie (*Pacing*) transmisji pakietów przy przeciążeniu odbiorcy

TCP – adresacja, numery portów



- ⌘ Numery portów - 16-bitowe liczby całkowite
- ⌘ Konieczne jest kontrolowanie ilości asocjacji w systemie operacyjnym – aby pula wolnych wartości portów TCP nie uległa wyczerpaniu
- ⌘ W architekturze klient-serwer - serwer zajmuje port TCP o danym numerze (zakłada „gniazdo”) i czeka na połączenia
- ⌘ Istnieją efemeryczne (krótkotrwałe) numery portów nadawane procesom-klientom przez moduł TCP na czas transmisji (aplikacja warstwy wyższej nie musi definiować ich wartości)
- ⌘ Istnieją liczne dedykowane (pre-definiowane) numery portów zwyczajowo łączone z konkretnymi usługami sieciowymi.
- ⌘ Strumień TCP jest jednokierunkowy – konieczne są dwa strumienie, aby utrzymać komunikację dwukierunkową. Strumienie te mogą korzystać z tego samego numeru portu.

Połączenia i asocjacje TCP/UDP



- ⌘ Połączenie TCP - ustanawia się między dwoma procesami w celu dwukierunkowego przesyłania danych.
- ⌘ Pół-asocjacja determinuje usługę (gniazdo TCP) i zawiera:
 - ⏏ Typ protokołu: TCP lub UDP
 - ⏏ Adres węzła oferującego usługę (liczba 32-bitowa)
 - ⏏ Numer portu lokalnego (liczba 16-bitowa)
- ⌘ Asocjacja - składa się z pięciu parametrów:
 - ⏏ Typ protokołu: TCP lub UDP (tu na potrzeby jednorazowego przesłania pakietu UDP)
 - ⏏ Adres węzła lokalnego (liczba 32-bitowa)
 - ⏏ Numer portu lokalnego (liczba 16-bitowa)
 - ⏏ Adres węzła zdalnego (liczba 32-bitowa)
 - ⏏ Numer portu obcego (liczba 16-bitowa)
- ⌘ Przykład asocjacji: (TCP, 128.0.0.2, 1612, 128.0.0.4, 80)

Gniazda TCP



- ⌘ Gniazdo zwane jest inaczej pół-asocjacją lub adresem transportowym
- ⌘ Zakładane przez serwer na danym porcie w celu odbioru połączeń przychodzących
- ⌘ Host może utrzymywać tylko jedno gniazdo o danym numerze portu, lecz może na jego bazie otworzyć wiele strumieni – także z innym, tym samym hostem
- ⌘ Dwa stany gniazda:
 - ☒ Pasywne czekanie
 - ☒ Właściwe połączenie
- ⌘ Możliwość odbioru wielu równoczesnych połączeń na dane gniazdo

1. *Journal of the American Medical Association*, 1997; 277: 1039-1043.

	4	8	12	16	20	24	28	32
Numer portu źródłowego				Numer portu docelowego				
Numer sekwencji								
Numer potwierdzenia								
Wyrów nanie	Rezerwa	Flagi		Rozmiar okna				

Budowa nagłówka TCP (I)

- ⌘ Port źródłowy TCP: 16-bitowe pole portu źródłowego przechowuje numer portu, który inicjuje sesje komunikacyjne. Port źródłowy i adres źródłowy IP funkcjonują jako adres zwrotny pakietu.
- ⌘ Port docelowy TCP: 16-bitowe pole portu docelowego jest adresem portu, dla którego przeznaczona jest transmisja. Port ten zawiera adres interfejsu aplikacji w komputerze odbiorcy, do której przesyłany jest pakiet danych.
- ⌘ Numer sekwencji TCP: 32-bitowy numer sekwencji jest wykorzystywany przez komputer odbierający do zrekonstruowania rozproszonych, rozbitych, podzielonych danych i przywrócenia im pierwotnej postaci. W sieci dynamicznie trasowanej może się zdarzyć, że niektóre pakiety pójdą innymi trasami i dotrą w niewłaściwej kolejności. Pole numeru sekwencji kompensuje tę niekonsekwencję w dostarczaniu.

Budowa nagłówka TCP (II)



- ⌘ Numer potwierdzenia TCP: TCP używa 32-bitowego potwierdzenia (ACK) pierwszego oktetu danych zawartego w następnym oczekiwanym segmencie. TCP może obliczyć ten numer, zwiększając numer ostatniego otrzymanego pakietu o liczbę oktetów w każdym segmencie TCP. Numer używany do identyfikowania każdego ACK jest numerem sekwencji potwierdzanego pakietu.
- ⌘ Wyrównanie danych: 4-bitowe pole przechowuje rozmiar nagłówka TCP, którego miarą jest 32-bitowa struktura danych, znana jako "słowo".
- ⌘ Zarezerwowane: 6-bitowe pole, zawsze ustawione na zero. Pole to jest zarezerwowane dla jeszcze nie wyspecyfikowanego, przyszłego zastosowania.

Budowa nagłówka TCP (III)

- ⌘ Flagi: 6-bitowe pole flagi zawiera sześć 1-bitowych flag, które umożliwiają realizację funkcji sterowania, takich jak pilność, zerowanie połączenia, synchronizacja numerów sekwencyjnych i zakończenie połączenia.
- ⌘ Rozmiar okna: 16-bitowe pole używane przez host docelowy w celu poinformowania komputera źródłowego o tym, ile danych jest gotów przyjąć w jednym segmencie TCP.
- ⌘ Suma kontrolna (16-bitów): Nagłówek TCP zawiera CRC. Host wysyłający oblicza wartość tego pola na podstawie zawartości segmentu. Host docelowy przeprowadza identyczne obliczenie. Jeśli zawartość pozostała nienaruszona, wynik obydwu obliczeń będzie identyczny, co świadczy o prawidłowości danych.
- ⌘ Wypełnienie: Do tego pola dodawane są zera, tak aby długość nagłówka TCP była zawsze wielokrotnością 32 bitów.

Cykl życia połączenia TCP (I)



- ⌘ 1. LISTEN- SŁUCHAJ, oczekiwanie na prośbę połączenia od jakiegokolwiek zdalnego TCP i portu.
- ⌘ 2. SYN-SENT- oczekiwanie na odpowiedź po wysłaniu prośby połączenia.
- ⌘ 3. SYN-RECEIVED - oczekiwanie na potwierdzenie połączenia (wcześniej wymieniono "prośby połączenia").
- ⌘ 4. ESTABLISHED- USTANOWIONE, reprezentuje otwarte połączenie, dane mogą być dostarczane do użytkownika. Normalny stan do transmisji danych.
- ⌘ 5. FIN-WAIT-1- oczekiwanie na prośbę zakończenia połączenia od zdalnego TCP lub potwierdzenie takiej prośby wcześniej wysłanej.

Cykl życia połączenia TCP (II)



- ⌘ 6. FIN-WAIT-2- oczekiwanie na prośbę zakończenia połączenia od zdalnego TCP.
- ⌘ 7. CLOSE-WAIT- oczekiwanie na prośbę zakończenia połączenia od lokalnego użytkownika.
- ⌘ 8. CLOSING - oczekiwanie na potwierdzenie zakończenia połączenia od zdalnego TCP.
- ⌘ 9. TIME-WAIT - czas przeznaczony na upewnienie się, że zdalny TCP otrzymał potwierdzenie zakończenia połączenia.
- ⌘ 10. CLOSED - połączenie nie istnieje.

TCP segmentacja



- ⌘ Strumień TCP dzielony jest na segmenty i takie porcje danych są umieszczane w ramkach. Numery sekwencji rosną zatem każdorazowo o wartość długości kolejnego segmentu lub jej wielokrotność.
- ⌘ Na wielkość segmentu wpływa cecha MSS (*Maximum Segment Size*) – znana stacji nadawczej.
- ⌘ Możliwe jest przesyłanie krótszego segmentu (końcówka danych, wymuszenie przez warstwę wyższą natychmiastowego wysłania danych krótszych itp.).
- ⌘ W celu optymalizacji transmisji długość powinna być obliczona na podstawie minimalnego MTU w warstwie drugiej ISO OSI na trasie transmisji, tak aby w pakiecie IP mieściła się wielokrotność MSS. Gdy MSS będzie zbyt mały – niepotrzebnie rośnie narzut danych związanych z obsługą fragmentacji.

Ustalanie MSS dla TCP

- ⌘ Istnieje możliwość ustalenia MSS dla ścieżki - PMTUD (*Path MTU Discovery*) – poprzez wykrycie MTU dla całej ścieżki od nadawcy do odbiorcy.
- ⌘ Procedura polega na wielokrotnym wysyłaniu testowego datagramu IP o konsekwentnie zmniejszanej długości.
- ⌘ Datagram dodatkowo posiada postawioną flagę DF (*Don't Fragment*). Gdy ruter otrzyma taki datagram i dodatkowo następny segment sieci na ścieżce posiada za małe MTU aby zmieścić datagram – ruter likwiduje go, a do nadawcy wysyła pakiet ICMP: *Fragmentation Required*.
- ⌘ Gdy kolejny wysłany datagram IP (o mniejszej długości) – nie spowoduje otrzymania komunikatu ICMP od któregokolwiek z ruterów na ścieżce – znany jest MTU dla ścieżki.
- ⌘ MSS obliczany jest poprzez pomniejszenie MTU o długości nagłówków IP, TCP i ew. innych protokołów.

TCP, buforowanie danych

- ⌘ U nadawcy (kolejno według rosnących wartości numerów sekwencji):
 - ⏏ Bufor danych już potwierdzonych (do kasacji)
 - ⏏ Bufor danych nadanych, lecz nie potwierdzonych (możliwa konieczność retransmisji)
 - ⏏ Bufor danych zgłoszonych przez aplikację wysyłającą do nadania
- ⌘ U nadawcy (kolejno według rosnących wartości numerów sekwencji):
 - ⏏ Bufor danych już wysłanych do aplikacji odbierającej (do kasacji)
 - ⏏ Bufor danych kompletnych, potwierdzonych nadawcy (ACK), lecz jeszcze nie pobranych przez aplikację odbierającą
 - ⏏ Bufor danych niekompletnych (oczekiwanie na komplet segmentów) – te dane mogą być częściowo potwierdzone komunikatem SACK
 - ⏏ Bufor pusty (następne okno zgłoszone nadawcy do zapełnienia)

TCP, buforowanie

- ⌘ Ramka potwierdzenia zawiera numer sekwencji pierwszego bajtu strumienia w formatowanym właśnie segmencie do odebrania (kolejny „potrzebny”)
- ⌘ Ramka potwierdzenia (ACK) może dotyczyć kilku poprzednich kolejnych segmentów jednocześnie (stwierdzając poprawność ich odbioru)
- ⌘ Gdy odbiorca otrzyma ACK z danym numerem sekwencji – może kasować dane z bufora nadawczego
- ⌘ Możliwe jest stosowanie systemu SACK (*Selective ACK*) – wtedy w polu „opcji” datagramu-potwierdzenia TCP przesyłana jest dodatkowa informacja zawierająca potwierdzenia odebrania wybranych SEGMENTÓW. Pozwala to na wykonanie selektywnej retransmisji – jedynie tych segmentów, których brakuje aby dane o adresie niższym niż numer sekwencji poprzedniego ACK uznać u nadawcy za dostarczone (co dzieje się w momencie ponownego otrzymania „pełnego” ACK, zamiast SACK)

Transmisja TCP – kontrola procesu nadawania

- ⌘ Dane wysyłane są ze stacji nadającej w wykorzystaniem algorytmów optymalizujących prędkość transmisji (z założeniem poszanowania wielkości okna przesłanej przez odbiorcę). Pula algorytmów (strategii) optymalizujących jest otwarta (slow start, Reno, Tahoe itp.).
- ⌘ Przy nadawaniu możliwe jest wymuszenie wysłania segmentu, nawet jeżeli ilość danych wysłanych przez aplikację nie przekroczyła MSS (typowa wartość MSS to 1460 – zakładająca funkcjonowanie TCP nad Ethernet). Taka funkcjonalność (określana słowami *push* lub *flush*) jest konieczna dla wsparcia kontrolowanego w czasie strumieniowania danych lub uzyskania natychmiastowej reakcji aplikacji z przeciwległej strony.
- ⌘ Jeśli bufor jest niepełny – jego zawartość najczęściej jest i tak rutynowo wysyłana po upływie 200 ms. Ta wartość jest jednak konfigurowalna. Możliwe jest także usunięcie tej cechy.

Transmisja TCP – Okno, congestion control



- ⌘ Okno to wielkość zadeklarowanego przez odbiorcę bufora danych przygotowanego do ich przyjęcia.
- ⌘ Aktualna wielkość okna jest konsekwentnie przekazywana do nadawcy w ramach potwierdzeń, regulując prędkości transmisji (*Congestion control*). Natywna maksymalna wartość – 64K (16 bitów)
- ⌘ Pole opcji w SYN może zawierać komendę skalowania okna (*TCP Window Scaling*) - bitowy SHL wartości max o 14 bitów. Wtedy okno może mieć max 1GB.
- ⌘ Wartość 0 powoduje wstrzymanie transmisji. Wartość ta jest przekazywana rutynowo gdy 32-bitowy numer sekwencji przelewa się.
- ⌘ Aby nie doszło do zablokowania protokołu po utracie polecenia wznowienia transmisji (kolejna ramka potwierdzenia została zgubiona) – nadawca wysyła tzw. mały pakiet (próbny) – co spowoduje wysłanie nowego potwierdzenia z nową wielkością okna (czyli zezwolenia na nadawanie określonej ilości danych)

Timery TCP (I)

- ⌘ RTO – *Retransmission TimeOut* – każdy pakiet wychodzący (opatrzone numerem sekwencji) uruchamia swój timer RTO. Gdy potwierdzenie pakietu odebrania nie dojdzie przed upłynięciem czasu timera – realizowana jest automatyczna retransmisja. Wartość RTO jest adaptowana dynamicznie na podstawie zmiennej SRTT (*Smoothed Round-Trip Time*), będącej bieżącą estymatą RTT do hosta zdalnego. Każde kolejne odebrane potwierdzenie umożliwia pomiar bieżącego RTT. Wtedy SRTT jest korygowany według formuły:
$$SRTT = 7/8 SRTT + 1/8 RTT \text{ (lub formuły zbliżonej).}$$

Później wprowadzono także wersję RTO inicjalizowaną wartościami uwzględniającymi zmienność RTT (powstała RTTVAR, *Round-Trip Time VARIation*), np. według formuły :

$$RTTVAR = 3/4 RTTVAR + 1/4 |SRTT - RTT|$$

Timery TCP (II)

- ⌘ *Persistence Timer* – służy do wyprowadzenia komunikacji z impasu (deadlock) powstałego po wyzerowaniu przez odbiorcę żądanej wielkości okna i utracie wysłanego przez niego datagramu pozwalającego na wznowienie ruchu (stacje będą wtedy czekać na siebie wzajemnie w nieskończoność). Jest uruchamiany po otrzymaniu potwierdzenia zawierającego wielkość okna równą 0 (polecenie wstrzymania transmisji). Gdy upłynie – wysyłany jest mały pakiet próbny (*window probe*)
- ⌘ *Keepalive Timer* – kiedy połączenie jest nieaktywne przez dłuższy czas (*Keepalive Timer* upływa) implementacja TCP może wykonać próbę obecności połączenia w hoście zdalnym i zamknąć połączenie w przypadku jej niepowodzenia
- ⌘ *Time Wait* – używany podczas zamykania połączenia, gdy upłynie – połączenie jest zamykane bezwarunkowo (zgubiono datagramy kończące połączenie w rutynowy sposób)

Strategie TCP, czyli manipulowanie wielkością okna

- ⌘ Potwierdzenie otrzymane od odbiorcy zawiera akceptowalną ilość danych, jaką można maksymalnie wysłać. Można jednak wysłać mniej, gdy niewielka przepustowość sieci grozi przeciążeniem.
- ⌘ Strategia *Slow Start* :
Po ustanowieniu połączenia, nadawca ustala wielkość okna jako wielkość bardzo małą - maksymalnie $4 \times$ wielkość segmentu TCP (mieszczącego się zawsze wewnątrz datagramu IP). Dodatkowo definiowana jest wartość parametru progu przepustowości (*threshold*), najczęściej kilkukrotnie większa od początkowej wielkości okna. Następnie z każdym odebraniem potwierdzenia TCP wielkość okna jest podwajana (i dwa razy więcej pakietów jest wysyłane w ciągu każdego RTT – *Round-Trip Time*). Gdy wielkość okna osiągnie *threshold* lub gdy w zaciną się problemy z komunikacją (brak potwierdzeń, sieć przeciążona) – do wartości *threshold* wpisywana jest **połowa** bieżącej wielkości okna, wielkość okna jest ponownie ustawiana jako bardzo małą i proces zaczyna się od nowa.

Strategie TCP (II)

⌘ Strategia *Tahoe*:

Po ustanowieniu połączenia, nadawca ustala wielkość okna jako wielkość bardzo małą - maksymalnie $4 \times$ wielkość segmentu TCP (mieszczącego się zawsze wewnątrz datagramu IP). Dodatkowo, tak samo jak w przypadku *Slow Start* definiowana jest wartość parametru progu przepustowości (*threshold*), najczęściej kilkukrotnie większa od początkowej wielkości okna. Następnie z każdym odebraniem potwierdzenia TCP wielkość okna jest podwajana (*multiplicative increase*) i dwa razy więcej pakietów jest wysyłane w ciągu każdego RTT. Gdy wielkość okna osiągnie *threshold* - dalej jest zwiększana, lecz już tylko z prędkością jednego segmentu na RTT (*additive increase*). Gdy w końcu zaczną się problemy (brak potwierdzeń) – do wartości *threshold* wpisywana jest **połowa** bieżącej wielkości okna, wielkość okna jest ponownie ustawiana jako bardzo mała i proces zaczyna się od nowa.

Strategie TCP (III)

⌘ Strategia *Reno*:

Po ustanowieniu połączenia, nadawca ustala wielkość okna jako wielkość bardzo małą - maksymalnie $4 \times$ wielkość segmentu TCP (mieszczącego się zawsze wewnątrz datagramu IP). Dodatkowo, tak samo jak w przypadku *Slow Start* definiowana jest wartość parametru progu przepustowości (*threshold*), najczęściej kilkukrotnie większa od początkowej wielkości okna. Następnie z każdym odebraniem potwierdzenia TCP wielkość okna jest podwajana i dwa razy więcej pakietów jest wysyłane w ciągu każdego RTT. Gdy wielkość okna osiągnie *threshold* - dalej jest zwiększana, lecz już tylko z prędkością jednego segmentu na RTT (*additive increase*). Gdy w końcu zaczną się problemy (brak potwierdzeń) – do wartości *threshold* i jednocześnie do wielkości okna wpisywana jest **połowa** bieżącej wielkości okna (*multiplicative decrease*). Od tej pory już zawsze wielkość okna rośnie tylko z prędkością jednego segmentu na RTT (*additive increase*) - lecz proces nie zaczyna ponownie od niskiej wartości wielkości okna.

Protokół UDP



- ⌘ *UDP - User Datagram Protocol*
- ⌘ Zapewnia proste i mające niewielki narzut kontrolny transmisje danych w pakietach o limitowanej długości
- ⌘ Nie dostarcza funkcjonalności w zakresie fragmentacji/defragmentacji w warstwie IV – pakiet UDP musi zmieścić się w datagramie IP
- ⌘ Brak funkcjonalności połączeniowej (protokół bez połączeniowy)
- ⌘ Zastosowanie: komunikaty systemowe, dane synchronizujące stan aplikacji, dane nie wymagające potwierdzeń

Budowa nagłówka UDP (II)

4	8	12	16	20	24	28	32
Numer portu źródłowego				Numer portu docelowego			
Długość				Suma kontrolna			

- ⌘ Numer portu źródłowego UDP: Port źródłowy jest numerem połączenia w komputerze źródłowym. Port źródłowy i adres źródłowy IP funkcjonują jako adres zwrotny pakietu.
- ⌘ Numer portu docelowego UDP: Port docelowy jest numerem połączenia w komputerze docelowym. Port docelowy UDP jest wykorzystywany do przekazywania pakietu odpowiedniej aplikacji, po tym jak pakiet dotrze do komputera docelowego.

Budowa nagłówka UDP (III)



- ⌘ Suma kontrolna UDP: Suma kontrolna jest polem kontroli błędów, którego wartość jest obliczana na podstawie zawartości segmentu. Komputer docelowy wykonuje taką samą funkcję matematyczną jak komputer źródłowy. Niezgodność dwóch obliczonych wartości wskazuje na wystąpienie błędu podczas transmisji pakietu.
- ⌘ Długość komunikatu UDP: Pole długości komunikatu informuje komputer docelowy o jego rozmiarze. Daje to komputerowi docelowemu kolejny mechanizm, wykorzystywany do sprawdzania poprawności wiadomości

Zalety UDP względem TCP

- ⌘ Ponieważ nie istnieją mechanizmy wymagające komunikacji zwrotnej możliwe jest:
 - ☒ Tworzenie uproszczonych implementacji UDP nie zawierających jednocześnie procedur odbioru i nadawania ramki UDP (co jest szczególnie istotne przy platformach ograniczonych wydajnościowo czy pamięciowo)
 - ☒ Broadcast ramki UDP – do wszystkich hostów w tym samym segmencie sieci
 - ☒ Zagwarantowanie odbioru wyłącznie całego komunikatu w danej chwili czasowej (nie wydarzy się sytuacja, w której do aplikacji przekazano fragment wysłanego przez inną aplikację datagramu UDP z założeniem, że jego dalsza część dopiero nadejdzie)
 - ☒ Natychmiastowa transmisja danych (w pierwszym datagramie)
 - bez konieczności przekazywania informacji wstępnej *3-way handshake* (SYN->SYN-ACK->ACK)

Zalety TCP względem UDP



- ⌘ Niezawodność – sytuacja, w której dane zaginęły po drodze od nadawcy do odbiorcy, zostanie wykryta. W konsekwencji aplikacja klienta nie musi utrzymywać kopii przekazanych już danych na wypadek zaistnienia konieczności ich retransmisji
- ⌘ Uporządkowana kolejność – w przeciwieństwie do serii datagramów UDP, kolejność segmentów TCP zostanie poprawnie odzwierciedlona w materiale odbieranym
- ⌘ Strumieniowanie – przekaz dowolnej długości danych niezależnie od funkcjonalności dolnych warstw ISO OSI
- ⌘ Utrzymywanie statusu połączenia – z możliwością sprawdzania danych o połączeniu w czasie rzeczywistym
- ⌘ Kontrola przepływu z ochroną węzłów mniej wydajnych obliczeniowo

IPX



- ⌘ IPX - *Internet Packet Exchange*
- ⌘ Protokół równoległy do TCP/IP
- ⌘ Bezpołączeniowy protokół operujący na datagramach
- ⌘ Operuje na warstwie sieciowej modelu ISO OSI. Adresuje i przesyła pakiety pomiędzy węzłami w obrębie sieci wielo segmentowej
- ⌘ Bazuje na polach adresowych znajdujących się w nagłówkach datagramów i na informacji otrzymanych z protokołów RIP
- ⌘ SPX (*Sequenced Packet Exchange*) - bazujący na IPX protokół warstwy wyższej zapewniający kontrolę „połączenia” i potwierdzanie pakietów (kontrola błędów).

Adresowanie w IPX



- ⌘ Kompletny sieciowy adres IPX ma postać dwunastobajtowej liczby heksadecymalnej
- ⌘ Serwery IPX - przechowują procesy IPX, adresowane docelowo w pakietach
- ⌘ Składowe adresu
 - Czterobajtowy numer sieci (adres serwera)
 - Sześciobajtowy numer hosta (adres serwera)
 - Dwubajtowy numer gniazda (numer procesu w serwerze)
- ⌘ Numer sieci jest stosowany przy routingu, numer węzła przy transmisjach lokalnych. Numery węzłów w różnych sieciach mogą się powtarzać.
- ⌘ Istnieją predefiniowane numery gniazd, pełniące z góry ustalone funkcje techniczne

IPX - struktura pakietu



⌘ Elementy:

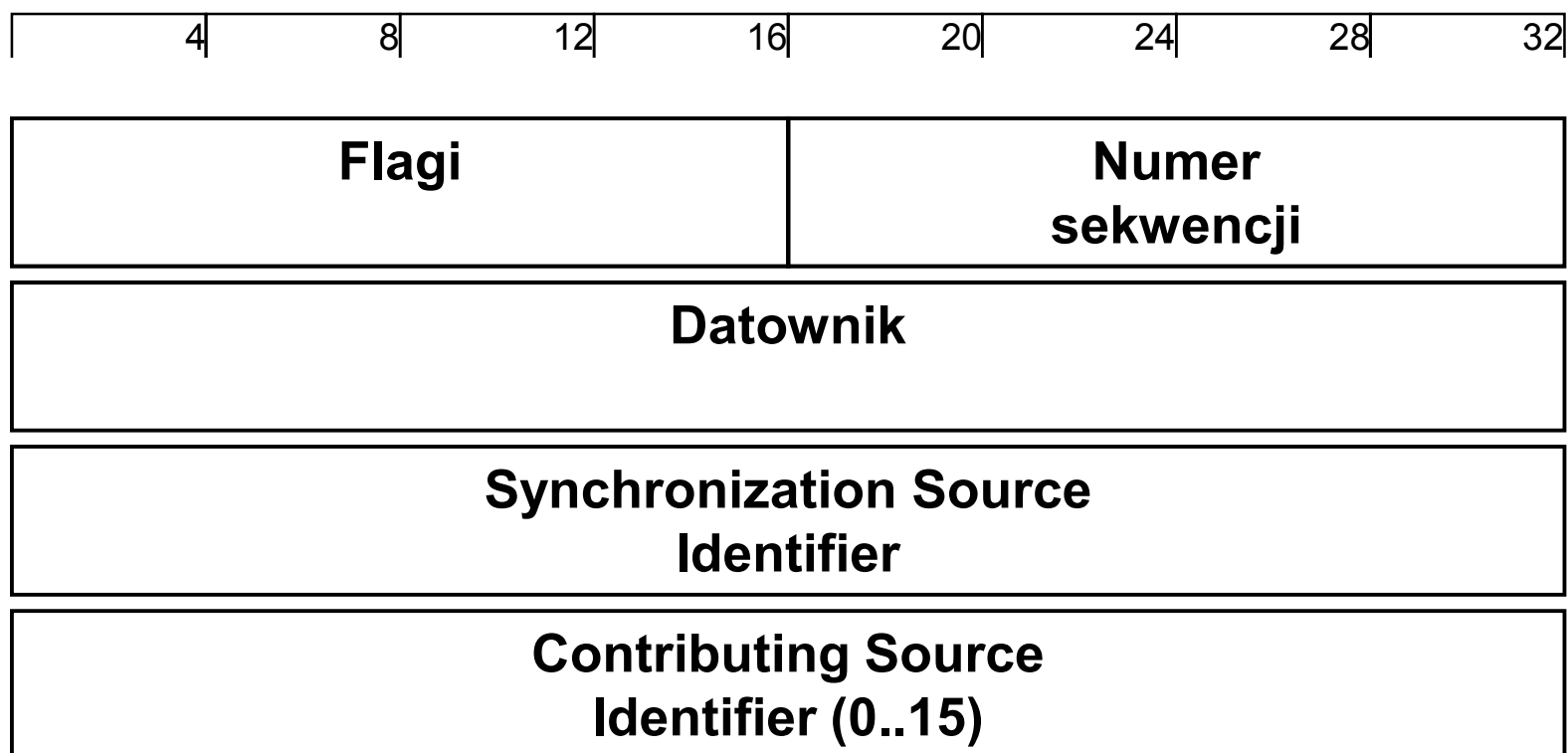
- ☒ 30-to bajtowy nagłówek IPX (*IPX Header*)
- ☒ Dane (w nich np. nagłówek SPX - protokół wyższy)
- ⌘ Minimalny rozmiar pakietu IPX - 30 bajtów , maksymalny - 65 535 bajtów
- ⌘ Nagłówek oprócz adresów źródłowego i docelowego posiada pole przedawnienia RIP (czyli maksymalnie 15 hopów), sumę kontrolną, pole długości, pole typu pakietu.
- ⌘ Typ pakietu to między innymi RIP, SPX, NetBIOS

RTP



- ⌘ RTP - *Real-time Transport Protocol*
- ⌘ Protokół obsługujący doręczanie danych czasu rzeczywistego
- ⌘ Główne zastosowania: interaktywne audio i video
- ⌘ Datagramy RTP są osadzone wewnątrz UDP (więc porty UDP specyfikują aplikacje-odbiorców)
- ⌘ Możliwy jest przesył danych do wielu miejsc docelowych (w wariancie IP multicast)
- ⌘ Protokół umożliwia opisywanie danych czasem nadania, scalanie różnych rodzajów danych w jedną porcję informacji (np. na użytek transmisji multimedialnych sygnałów zespolonych), czy sekwencyjne numerowanie przesyłanych pakietów
- ⌘ Brak gwarancji dostarczenia na czas, brak mechanizmów retransmisji (jeśli potrzebne, aplikacja klienta musi je implementować)

Budowa datagramu RTP (I)



Budowa datagramu RTP (II)

- ⌘ Numer sekwencji (*Sequence number*, 16 bitów) - inkrementowany o 1 dla każdego wysyłanego pakietu danych RTP. Służy do kontroli kolejności ramek. Początkowa wartość numeru jest losowa.
- ⌘ Datownik, (*Timestamp*, 32 bity) - chwila próbkowania pierwszego bajtu w pakiecie danych RTP. Chwila próbkowania zależy od zegara inkrementowanego liniowo w czasie. Początkowa wartość datownika jest losowa. Wiele kolejnych pakietów RTP może mieć ten sam znacznik czasowy, jeśli są równocześnie generowane (np. należą do tej samej ramki video). Kolejne pakiety mogą nie mieć rosnących znaczników, jeśli nie są transmitowane w porządku, w jakim były próbkowane.
- ⌘ Źródło synchronizowanych danych (*Synchronization source identifier*, 32 bity) - identyfikator jest nadawany losowo dla składowej strumienia zespolonego w sygnale (może istnieć wiele strumieni zespolonych)
- ⌘ Lista *Contributing source identifier* (0 do 15 pozycji po 32 bity) – informacje dodatkowo opisujące dane zawarte w pakiecie

Budowa datagramu RTP (III)

⌘ Flagi (CD)

- ☒ CC (4 bity) – pole identyfikujące liczbę synchronizowanych strumieni w danym sygnale zespolonym (każdy pakiet należy do któregoś z nich). Pole jest wykorzystywane głównie przez miksery audio w określeniu odpowiednich natężeń sygnałów przy miksowaniu. Przynależność pakietu do konkretnego strumienia jest określana przez identyfikator *Synchronization source identifier*.
- ☒ Version (2 bity) – wersja RTP
- ☒ Typ danych (*Payload Type*, 8 bitów) – identyfikator określający, jakiego algorytmu należy użyć do rozkodowania danych multimedialnych. System kodowania może się zmienić podczas transmisji – na przykład w konsekwencji wykrycia pogorszenia się jakości łącza (gdyż każdy pakiet posiada to pole)

Budowa datagramu RTP (II)

⌘ Flagi

- ☒ *P (padding)* – długość pakietu została sztucznie powiększona do pełnych 32 bitów
- ☒ *M (application specific marker bit)* – szczególnie istotny, oznacza pakiet zawierający początek wydzielonego fragmentu multimedialnego - w rozumieniu aplikacji z nich korzystającej, np. próbki dźwiękowej wysłanej po okresie ciszy (próbka taka określana jest mianem *talkspurt*), zespolonej z dźwiękiem ramki wideo itp.
- ☒ *X (extension header)* – załączono dodatkowy nagłówek opisujący dane (wewnątrz danych UDP znajduje się nagłówek RTP i jeszcze dodatkowe informacje opisujące treść). W tej sytuacji pole *payload type* nie musi być wiążące. Pierwsze 16 bitów pola *extension header* to jego długość, reszta nie jest wyspecyfikowana (może służyć do tworzenia własnych protokołów)

Aplikacje odtwarzające dane RTP

- ⌘ Ponieważ pakiety mogą przychodzić z opóźnieniami, konieczne jest przesunięcie czasu odtwarzania bufora odebranych danych (poprzez ustalenie tzw. *playback point* – tolerowanego opóźnienia odtwarzania, przy którym spóźnienie pakietu jest już tylko incydentalne)
- ⌘ Pakiety mogą osiągać cel z różnymi opóźnieniami (rozrzut opóźnienia określa się terminem *jitter*). Do obliczenia opóźnienia procesu odtwarzania danych stosuje się także *jitter* - obliczany i aktualizowany (w przypadku użycia RTP) na podstawie różnicy wartości pola *timestamp* i faktycznego czasu otrzymania pakietu
- ⌘ Dwie strategie stosowane przy odtwarzaniu multimediiów, gdy pakiet nie przyszedł na czas:
 - ⏏ Chwilowe zatrzymanie (*playback gap*) – stosowane najczęściej przy odtwarzaniu multimediiów składowanych wcześniej w hoście zdalnym
 - ⏏ Pominięcie (media skip) z interpolacją, jeśli możliwa – stosowane głównie przy sygnałach multimedialnych czasu rzeczywistego

Aplikacje odtwarzające dane RTP, RTCP



- ⌘ Dodatkowym udogodnieniem jest możliwość zastosowania protokołu RTCP (Real-time Control Protocol) - uzupełniającego funkcjonalność RTP
- ⌘ RTCP nie transportuje danych multimedialnych, a jedynie informację dodatkową (kontrolną). W swoich raportach (*RTCP report*) przekazuje on informacje wspomagającą pozwalającą na:
 - ⌘ - eliminowanie problemów sieci (jitter, delay, bandwidth itp.) – przesyłając raporty dane także w kierunku nadawcy (co pozwala np. na zmianę systemu kodowania, korektę jakości itp.)
 - ⌘ - synchronizację wielu strumieni zespolonych (*Interstream synchronization*)
- ⌘ Samo-regulację ilości raportów przy ruchu multicast (np. 5 % faktycznego strumienia danych multimedialnych)
- ⌘ Opisywanie multimediiów (udostępniane użytkownikowi)

Inne protokoły warstwie transportowej



- ⌘ Poniższe rozwiązania są niszowe i w większości wymagające modyfikowania socket API lub innych funkcjonalności systemowych, jednak zostały formalnie zdefiniowane, posiadają implementacje i mogą się w przyszłości spopularyzować
- ⌘ SCTP (Stream Control Transmission Protocol) oraz SST (Structured Stream Transport) – umożliwiają ustanawianie strumieni powiązanych relacjami, powiązanych ze sobą jednoczesnych transmisji Connection-oriented i connectionless) czy powiązanych strumieni prowadzonych jednocześnie do wielu lokalizacji
- ⌘ DCCP (Datagram Congestion Controlled Protocol) – modyfikacja UDP pozwalająca na sterowanie przepływem